

Prometheus Setup Guide for Spring Boot

1. Objective

Integrate Prometheus with a Spring Boot application using Spring Boot Actuator and Micrometer, verify that Prometheus scrapes application metrics successfully, and prepare the application for Grafana integration.

2. Architecture

React UI → Spring Boot → Spring Boot Actuator + Micrometer → Prometheus

Spring Boot exposes metrics at **/actuator/prometheus**. Prometheus periodically pulls (scrapes) these metrics every 15 seconds.

3. Maven Dependencies

Add the following dependencies:

- spring-boot-starter-actuator
- micrometer-registry-prometheus

Actuator exposes operational endpoints while Micrometer converts Spring Boot metrics into the Prometheus format.

4. Configure application.properties

```
management.endpoints.web.exposure.include=health,info,prometheus
management.endpoint.prometheus.access=unrestricted
management.metrics.tags.application=Employee-Management-System
```

These properties expose the Prometheus endpoint and assign a common application tag to all metrics.

5. Verify Spring Boot Metrics

Start the application and open:
http://localhost:8080/actuator/prometheus

You should see hundreds of metrics such as:

- jvm_memory_used_bytes
- system_cpu_usage
- http_server_requests_seconds_count
- process_uptime_seconds

6. Create prometheus.yml

```
global:
  scrape_interval: 15s
```

```
scrape_configs:
```

```
- job_name: 'spring-boot'
  metrics_path: '/actuator/prometheus'
  static_configs:
    - targets: ['host.docker.internal:8080']
```

Since Prometheus runs inside Docker while Spring Boot runs on Windows, use **host.docker.internal** instead of localhost.

7. Run Prometheus using Docker

```
docker pull prom/prometheus
```

```
docker run -d --name prometheus -p 9090:9090 -v
C:\prometheus\prometheus.yml:/etc/prometheus/prometheus.yml
prom/prometheus
```

8. Verify Prometheus

Open <http://localhost:9090>.

Navigate to **Status** → **Target Health**.

The target **spring-boot** should be in the **UP** state.

9. Verify Using PromQL

Run the following queries:

- `up` → Returns 1 if Prometheus can reach Spring Boot.
- `jvm_memory_used_bytes` → JVM memory usage.
- `http_server_requests_seconds_count` → Total HTTP requests.
- `system_cpu_usage` → Current CPU utilization.
- `jvm_threads_live_threads` → Live JVM threads.
- `process_uptime_seconds` → Application uptime.

10. Results Observed

During implementation the following were verified successfully:

- Spring Boot metrics endpoint accessible.
- Prometheus Docker container running.
- spring-boot target status = UP.
- Query 'up' returned value 1.
- JVM, CPU and HTTP metrics were successfully retrieved.

11. Troubleshooting

- If Target = DOWN, verify Spring Boot is running.
- Verify `/actuator/prometheus` is accessible.
- Ensure `prometheus.yml` uses `host.docker.internal`.
- Verify Docker volume mapping.
- Check 'docker logs prometheus' for errors.

12. Conclusion

The Spring Boot application is successfully integrated with Prometheus. Prometheus continuously scrapes metrics from the application, enabling monitoring of JVM, CPU, HTTP requests, uptime, database connection pool and future custom business metrics.